

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 2**



ANDROID LAYOUT WITH COMPOSE

Oleh:

Naufal Elyzar NIM. 231017210019

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
APRIL 2024**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 2

Laporan Praktikum Pemrograman Mobile Modul 2: Android Layout With Compose ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Naufal Elyzar
NIM : 2310817210019

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Salsabila Syifa
NIM. 2010817320004

Ir. Eka Setya Wijaya, S.T., M.Kom.
NIP. 198205082008011010

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	7
B. Output Program.....	16
C. Pembahasan	17
D. Tautan Git	18
SOAL 2.....	18
A. Pembahasan	19

DAFTAR GAMBAR

Gambar 1 Screenshot Hasil Jawaban Soal 1 XML	16
Gambar 2 Screenshot Hasil Jawaban Soal 1 Jetpack Compose	16

DAFTAR TABEL

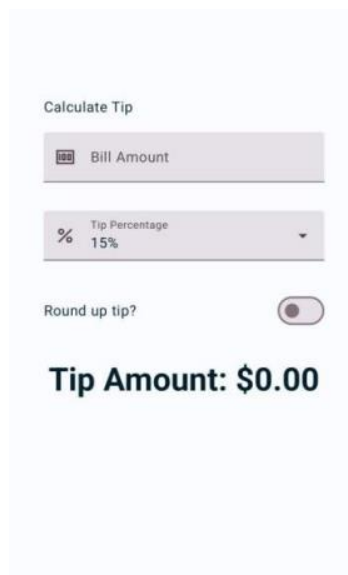
Tabel 1 Source Code Jawaban Soal 1.....	12
Tabel 2 Source Code Jawaban Soal 1.....	13
Tabel 3. Source Code Jawaban Soal 1.....	15

Soal Praktikum:

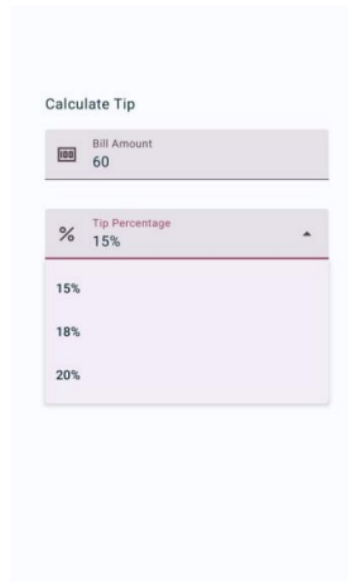
SOAL 1

Buatlah sebuah aplikasi kalkulator tip menggunakan XML dan Jetpack Compose yang dirancang untuk membantu pengguna menghitung tip yang sesuai berdasarkan total biaya layanan yang mereka terima. Fitur-fitur yang diharapkan dalam aplikasi ini mencakup:

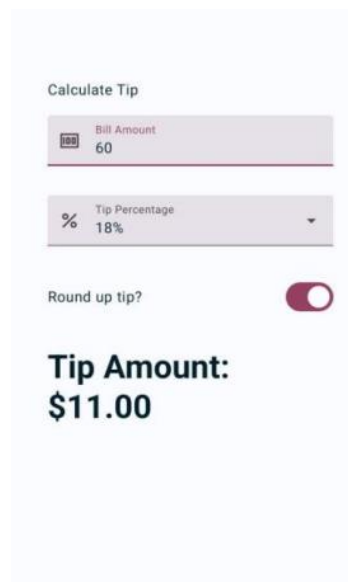
- Input biaya layanan: Pengguna dapat memasukkan total biaya layanan yang diterima dalam bentuk nominal.
- Pilihan persentase tip: Pengguna dapat memilih persentase tip yang diinginkan.
- Pengaturan pembulatan tip: Pengguna dapat memilih untuk membulatkan tip ke angka yang lebih tinggi.
- Tampilan hasil: Aplikasi akan menampilkan



Gambar 1 Tampilan Awal Aplikasi



Gambar 2 Tampilan Pilihan Persentase Tip



Gambar 3 Tampilan Aplikasi Setelah Dijalankan

A. Source Code

MainActivity.kt (Compose)

1	package com.example.tiptime
2	
3	import android.os.Bundle

```

4 import androidx.activity.ComponentActivity
5 import androidx.activity.compose.setContent
6 import androidx.activity.enableEdgeToEdge
7 import androidx.annotation.DrawableRes
8 import androidx.annotation.StringRes
9 import androidx.compose.foundation.layout.*
10 import androidx.compose.foundation.rememberScrollState
11 import androidx.compose.foundation.text.KeyboardOptions
12 import androidx.compose.foundation.verticalScroll
13 import androidx.compose.material3.*
14 import androidx.compose.runtime.*
15 import androidx.compose.ui.Alignment
16 import androidx.compose.ui.Modifier
17 import androidx.compose.ui.res.painterResource
18 import androidx.compose.ui.res.stringResource
19 import androidx.compose.ui.text.input.ImeAction
20 import androidx.compose.ui.text.input.KeyboardType
21 import androidx.compose.ui.tooling.preview.Preview
22 import androidx.compose.ui.unit.dp
23 import com.example.tiptime.ui.theme.TipTimeTheme
24 import java.text.NumberFormat
25
26 class MainActivity : ComponentActivity() {
27     override fun onCreate(savedInstanceState: Bundle?) {
28         enableEdgeToEdge()
29         super.onCreate(savedInstanceState)
30         setContent {
31             TipTimeTheme {
32                 Surface(
33                     modifier = Modifier.fillMaxSize()
34                 ) {
35                     TipTimeLayout()
36                 }
37             }
38         }
39     }
40 }
41 @OptIn(ExperimentalMaterial3Api::class)
42 @Composable
43 fun TipTimeLayout() {
44     var amountInput by remember { mutableStateOf("") }
45     var roundUp by remember { mutableStateOf(false) }
46
47     val tipOptions = listOf("15%", "18%", "20%")
48     var tipExpanded by remember { mutableStateOf(false) }
49     var selectedTipOption by remember {
50         mutableStateOf(tipOptions[0]) }

```



```

50
51     val amount = amountInput.toDoubleOrNull() ?: 0.0
52     val                                     tipPercent                                     =
selectedTipOption.removeSuffix("%").toDoubleOrNull()?.div(100)
?: 0.0
53     val tip = calculateTip(amount, tipPercent, roundUp)
54
55     Column(
56         modifier = Modifier
57             .statusBarsPadding()
58             .padding(horizontal = 40.dp)
59             .verticalScroll(rememberScrollState())
60             .safeDrawingPadding(),
61         horizontalAlignment = Alignment.CenterHorizontally,
62         verticalArrangement = Arrangement.Center
63     ) {
64         Text(
65             text = stringResource(R.string.calculate_tip),
66             modifier = Modifier
67                 .padding(bottom = 16.dp, top = 40.dp)
68                 .align(alignment = Alignment.Start)
69         )
70
71         EditNumberField(
72             label = R.string.bill_amount,
73             leadingIcon = R.drawable.money,
74             keyboardOptions = KeyboardOptions.Default.copy(
75                 keyboardType = KeyboardType.Number,
76                 imeAction = ImeAction.Next
77             ),
78             value = amountInput,
79             onValueChanged = { amountInput = it },
80             modifier = Modifier
81                 .padding(bottom = 32.dp)
82                 .fillMaxWidth(),
83         )
84
85         ExposedDropDownMenuBox(
86             expanded = tipExpanded,
87             onExpandedChange = { tipExpanded = !tipExpanded }
88         ) {
89             TextField(
90                 readOnly = true,
91                 value = selectedTipOption,
92                 onValueChange = {},
93                 label = { Text("Tip Percentage") },
94                 leadingIcon = {

```

```

95             Icon(
96                 painter = painterResource(id =
R.drawable.percent),
97                 contentDescription = null
98             )
99         },
100         trailingIcon = {
101             ExposedDropdownMenuDefaults.TrailingIcon(expanded
= tipExpanded)
102         },
103         colors =
ExposedDropdownMenuDefaults.textFieldColors(),
104         modifier = Modifier
105             .menuAnchor()
106             .fillMaxWidth()
107     )
108
109     ExposedDropdownMenu(
110         expanded = tipExpanded,
111         onDismissRequest = { tipExpanded = false }
112     ) {
113         tipOptions.forEach { option ->
114             DropdownMenuItem(
115                 text = { Text(option) },
116                 onClick = {
117                     selectedTipOption = option
118                     tipExpanded = false
119                 }
120             )
121         }
122     }
123
124     Spacer(modifier = Modifier.height(24.dp))
125     RoundTheTipRow(
126         roundUp = roundUp,
127         onRoundUpChanged = { roundUp = it }
128     )
129     Spacer(modifier = Modifier.height(24.dp))
130     Text(
131         text = stringResource(id = R.string.tip_amount,
tip),
132         style = MaterialTheme.typography.headlineMedium
133     )
134 }
135 }
136 }
137

```

```

138 @Composable
139 fun EditNumberField(
140     @StringRes label: Int,
141     @DrawableRes leadingIcon: Int,
142     keyboardOptions: KeyboardOptions,
143     value: String,
144     onValueChanged: (String) -> Unit,
145     modifier: Modifier = Modifier
146 ) {
147     TextField(
148         value = value,
149         singleLine = true,
150         leadingIcon = { Icon(painter = painterResource(id =
leadingIcon), contentDescription = null) },
151         modifier = modifier,
152         onValueChange = onValueChanged,
153         label = { Text(stringResource(label)) },
154         keyboardOptions = keyboardOptions
155     )
156 }
157
158 @Composable
159 fun RoundTheTipRow(
160     roundUp: Boolean,
161     onRoundUpChanged: (Boolean) -> Unit,
162     modifier: Modifier = Modifier
163 ) {
164     Row(
165         modifier = modifier.fillMaxWidth(),
166         verticalAlignment = Alignment.CenterVertically
167     ) {
168         Text(text = stringResource(R.string.round_up_tip))
169         Switch(
170             modifier = Modifier
171                 .fillMaxWidth()
172                 .wrapContentWidth(Alignment.End),
173             checked = roundUp,
174             onCheckedChange = onRoundUpChanged
175         )
176     }
177 }
178
179 private fun calculateTip(amount: Double, tipPercent: Double,
roundUp: Boolean): String {
180     var tip = tipPercent * amount
181     if (roundUp) {
182         tip = kotlin.math.ceil(tip)

```

```

183     }
184     return NumberFormat.getCurrencyInstance().format(tip)
185 }
186
187 @Preview(showBackground = true)
188 @Composable
189 fun TipTimeLayoutPreview() {
190     TipTimeTheme {
191         TipTimeLayout()
192     }
193 }
194

```

Tabel 1 Source Code Jawaban Soal 1

MainActivity.kt (XML)

```

1 package com.example.modul2xmlbaru
2
3 import android.os.Bundle
4 import android.text.Editable
5 import android.text.TextWatcher
6 import android.view.View
7 import android.widget.*
8 import androidx.appcompat.app.AppCompatActivity
9 import java.text.NumberFormat
10 import kotlin.math.ceil
11
12 class MainActivity : AppCompatActivity() {
13
14     private lateinit var billAmountInput: EditText
15     private lateinit var tipPercentageDropdown:
16     AutoCompleteTextView
17     private lateinit var roundUpSwitch: Switch
18     private lateinit var tipResult: TextView
19
20     private val tipOptions = listOf("15%", "18%", "20%")
21
22     override fun onCreate(savedInstanceState: Bundle?) {
23         super.onCreate(savedInstanceState)
24         setContentView(R.layout.activity_main)
25
26         billAmountInput =
27         findViewById(R.id.billAmountInput)
28         roundUpSwitch = findViewById(R.id.roundUpSwitch)
29         tipResult = findViewById(R.id.tipResult)
30
31         billAmountInput.addTextChangedListener(inputWatcher)
32
33     }
34 }
35

```

```

30         roundUpSwitch.setOnCheckedChangeListener { _, _ ->
calculateTip() }
31         tipPercentageDropdown =
findViewById(R.id.tipPercentageDropdown)
32         val adapter = ArrayAdapter(this,
android.R.layout.simple_dropdown_item_1line, tipOptions)
33         tipPercentageDropdown.setAdapter(adapter)
34         tipPercentageDropdown.keyListener = null
tipPercentageDropdown.setOnItemClickListener { _,
35 _', _', _ ->
36         calculateTip()
37     }
38 }
39
40 private val inputWatcher = object : TextWatcher {
41     override fun beforeTextChanged(s: CharSequence?,
start: Int, count: Int, after: Int) {}
42     override fun afterTextChanged(s: Editable?) {}
43
44     override fun onTextChanged(s: CharSequence?, start:
Int, before: Int, count: Int) {
45         calculateTip()
46     }
47 }
48
49 private fun calculateTip() {
50     val amount =
billAmountInput.text.toString().toDoubleOrNull() ?: 0.0
51     val tipPercentText =
tipPercentageDropdown.text.toString()
52     val tipPercent =
tipPercentText.removeSuffix("%").toDoubleOrNull()?.div(100)
?: 0.0
53     var tip = amount * tipPercent
54
55     if (roundUpSwitch.isChecked) {
56         tip = ceil(tip)
57     }
58
59     val formattedTip =
NumberFormat.getCurrencyInstance().format(tip)
60     tipResult.text = getString(R.string.tip_amount,
formattedTip)
61 }
62 }

```

Tabel 2 Source Code Jawaban Soal 1

activity_main.xml

```

1  <ScrollView
    xmlns:android="http://schemas.android.com/apk/res/android"
2      xmlns:app="http://schemas.android.com/apk/res-auto"
3      android:layout_width="match_parent"
4      android:layout_height="match_parent">
5
6      <LinearLayout
7          android:id="@+id/layoutRoot"
8          android:layout_width="match_parent"
9          android:layout_height="wrap_content"
10         android:orientation="vertical"
11         android:padding="24dp"
12         android:gravity="center_horizontal">
13
14     <TextView
15         android:id="@+id/tvTitle"
16         android:layout_width="wrap_content"
17         android:layout_height="wrap_content"
18         android:text="@string/calculate_tip"
19         android:textSize="24sp"
20         android:textStyle="bold"
21         android:layout_marginBottom="16dp" />
22
23     <com.google.android.material.textfield.TextInputLayout
24         android:layout_width="match_parent"
25         android:layout_height="wrap_content"
26         android:layout_marginBottom="16dp"
27         android:hint="Bill Amount">
28
29         <com.google.android.material.textfield.TextInputEditText
30             android:id="@+id/billAmountInput"
31             android:layout_width="match_parent"
32             android:layout_height="wrap_content"
33             android:inputType="numberDecimal"
34             android:drawableStart="@drawable/money"
35             android:drawablePadding="8dp" />
36     </com.google.android.material.textfield.TextInputLayout>
37
38     <com.google.android.material.textfield.TextInputLayout
39         style="@style/Widget.Material3.TextInputLayout.OutlinedBox.
40         ExposedDropdownMenu"
41         android:layout_width="match_parent"
42         android:layout_height="wrap_content"
43         android:layout_marginBottom="16dp"
44         android:hint="Tip Percentage"
45         app:endIconMode="dropdown_menu">

```

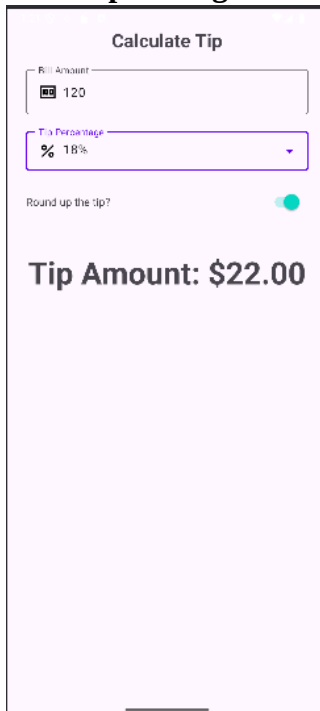
```

44
45 <AutoCompleteTextView
46         android:id="@+id/tipPercentageDropdown"
47         android:layout_width="match_parent"
48         android:layout_height="51dp"
49         android:drawableStart="@drawable/percent"
50         android:drawablePadding="8dp"
51         android:inputType="none"
52         android:paddingStart="16dp" />
53     </com.google.android.material.textfield.TextInputLayout>
54
55 <LinearLayout
56         android:layout_width="match_parent"
57         android:layout_height="wrap_content"
58         android:orientation="horizontal"
59         android:gravity="center_vertical"
60         android:layout_marginBottom="16dp">
61
62 <TextView
63         android:layout_width="0dp"
64         android:layout_weight="1"
65         android:layout_height="wrap_content"
66         android:text="Round up the tip?" />
67
68 <Switch
69         android:id="@+id/roundUpSwitch"
70         android:layout_width="wrap_content"
71         android:layout_height="wrap_content"
72         android:minWidth="48dp"
73         android:minHeight="48dp"
74         android:padding="8dp" />
75
76 </LinearLayout>
77
78 <TextView
79         android:id="@+id/tipResult"
80         android:layout_width="wrap_content"
81         android:layout_height="wrap_content"
82         android:layout_marginTop="24dp"
83         android:text="Tip Amount: $0.00"
84         android:textSize="40sp"
85         android:textStyle="bold" />
86
87 </LinearLayout>
88 </ScrollView>
89

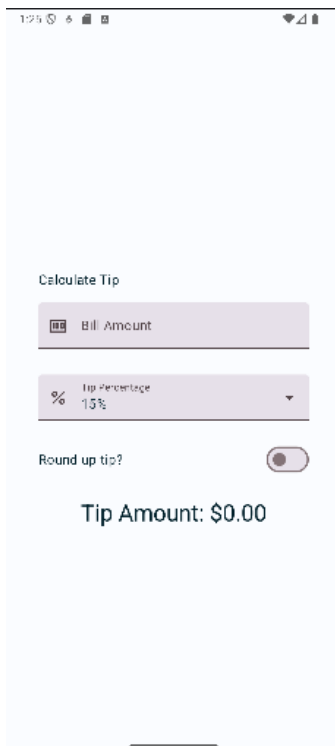
```

Tabel 3. Source Code Jawaban Soal 1

B. Output Program



Gambar 1 Screenshot Hasil Jawaban Soal 1 XML



Gambar 2 Screenshot Hasil Jawaban Soal 1 Jetpack Compose

C. Pembahasan

MainActivity.kt(Compose):

Pada line 1 digunakan untuk mendefinisikan nama package

Pada line 3-24, Library-library yang dibutuhkan pada proyek ini

Pada line 26, berguna untuk meng-override fungsi onCreate() yang sudah ada di superclass

Pada line 27, 'enableEdgeToEdge()' membuat konten tampil hingga ke tepi layar

Pada line 29, 'setContent {}' memulai UI dengan Jetpack Compose.

Pada line 32, 'Surface' dipakai untuk latar belakang.

Line 35 adalah fungsi yang menggambar UI utama.

Pada line 44 digunakan untuk menyimpan input jumlah tagihan dari user

Pada line 47 merupakan list pilihan persentase tip.

Pada line 52 berfungsi mengubah "15%" jadi 0.15 agar bisa dihitung.

Line 53 berguna untuk menghitung tip berdasarkan jumlah, persentase, dan apakah dibulatkan.

Pada line 55, 'Column' merupakan layout vertikal, semua isi di dalamnya ditumpuk ke bawah

Pada line 57, 'statusBarsPadding()' digunakan untuk menambah padding di atas supaya konten tidak menabrak status bar.

Pada line 58, 'padding(horizontal = 40.dp)' menambah padding kanan-kiri sebesar 40dp

Pada line 59, 'verticalScroll(rememberScrollState())' membuat konten bisa di-scroll ke bawah.

Pada line 60, 'safeDrawingPadding()' menambah padding supaya konten tidak ketutupan area "safe" seperti notch

Pada line 61, 'horizontalAlignment = Alignment.CenterHorizontally' membuat semua isi di dalam Column akan di tengah horizontal.

Pada line 62, 'verticalArrangement = Arrangement.Center' membuat isi Column akan diatur agar berkumpul di tengah secara vertikal.

Pada line 62-69 berfungsi untuk menampilkan teks judul dari strings.xml.

Pada line 71-83, Membuat input angka.

Pada line 85-123, membuat dropdown dengan pilihan 15%, 18%, 20%. 'TextField' diset 'readOnly' dan tampilkan icon persen di depan. Ketika user mengklik, muncul menu pilihan. Jika klik salah satu item: set ke 'selectedTipOption' dan tutup menu ('tipExpanded = false').

Pada line 125, memberi jarak vertical.

Pada line 126-129 berguna untuk menampilkan teks "Round up the tip?" dan switch di sebelah kanan. Kalau switch diaktifkan, nilai tip akan dibulatkan ke atas ('ceil().')

Pada line 26, digunakan fungsi setContent untuk mendefinisikan UI aplikasi menggunakan Jetpack Compose.

Pada line 131-134, menampilkan hasil kalkulasi tip dalam format uang.

Pada line 138-146, digunakan untuk membuat TextField (inputan teks) khusus angka

Pada line 147-156 merupakan UI dari kode diatas. Fungsinya yaitu menampilkan TextField satu baris dan menambahkan ikon, lalu keyboard hanya menampilkan angka.

Pada line 158-163 berguna untuk membuat baris dengan teks dan switch untuk mengaktifkan pembulatan tip.

Pada line 164-177, row artinya ditampilkan secara horizontal. Ketika switch di klik, fungsi 'onRoundUpChanged' akan dipanggil untuk mengubah state-nya.

Pada line 179-185 merupakan function untuk menghitung tip.

Pada line 187-193 berfungsi untuk menampilkan preview UI di Android Studio tanpa emulator

MainActivity.kt(Compose):

Pada baris 1 merupakan package aplikasi saya

Baris 3-10 merupakan library yang diperlukan pada projek aplikasi ini

Di baris 12, ini adalah kelas utama yang berjalan saat aplikasi dibuka

Baris 14-19, Properti ini di-lateinit karena baru diisi di 'onCreate()'. 'tipOptions:' merupakan daftar persen tip yang bisa dipilih user.

Dari baris 21-38 merupakan fungsi yang dipanggil saat aktivitas pertama kali dibuat. 'setContentView(R.layout.activity_main)' Menghubungkan MainActivity dengan layout 'XML activity_main.xml'. 'billAmountInput = findViewById(R.id.billAmountInput)' menghubungkan ID di XML dengan variabel Kotlin. 'billAmountInput.addTextChangedListener(inputWatcher)' agar input teks dan switch akan langsung memicu perhitungan ulang saat ada perubahan.

Baris 40-47 akan dipanggil saat user mengetik di input tagihan lalu memicu fungsi 'calculateTip()' secara otomatis.

Baris 49-62, 'val amount = billAmountInput.text.toString().toDoubleOrNull() ?: 0.0' akan mengambil nilai dari input dan diubah jadi Double. Kalau tidak valid. 'if (roundUpSwitch.isChecked) { 'Kalau switch aktif, bulatkan nilai tip ke atas pakai ceil(). Baris terakhir berfungsi untuk memformat hasil jadi mata uang local.

activity_main.xml:

Pada line 1-4, 'ScrollView:' berfungsi agar layout bisa di-scroll

Pada line 6-12, 'LinearLayout:' untuk menyusun elemen secara vertikal. gravity="center_horizontal": Semua isi akan di-center secara horizontal.

Pada line 14-21, berguna untuk menampilkan teks "Calculate Tip". Teks diambil dari strings.xml. Teksnya besar (24sp) dan tebal.

Pada line 23-36, digunakan untuk membuat input dengan style Material Design 3. 'TextInputLayout' berfungsi seperti pembungkus yang menampilkan label saat input aktif.

Pada line 38-54 merupakan dropdown untuk memilih persen tip (15%, 18%, 20%). 'AutoCompleteTextView:' adalah komponen utama dropdown-nya. 'android:inputType="none":' supaya tidak muncul keyboard saat diklik.

Pada line 56-77 berfungsi agar susunannya horizontal.

Pada line 79-88, Digunakan untuk menampilkan hasil perhitungan tip.

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/NaufalElyzar11/PraktikumMobile/tree/main/Modul%202>

SOAL 2

Jelaskan perbedaan dari implementasi XML dan Jetpack Compose beserta kelebihan dan kekurangan dari masing-masing implementasi.

A. Pembahasan

Dalam membangun UI Android, Jetpack Compose dan XML berbeda. XML membagi layout dan kotlin, sehingga terkadang membutuhkan kode tambahan seperti `findViewById()` atau `ViewBinding`. Sebaliknya, Compose menyatukan tampilan dan logika dalam satu file Kotlin, yang membuat kode lebih mudah dan reaktif terhadap perubahan data. Menurut saya XML membutuhkan waktu yang lebih lama dalam pembuatannya. Kelebihannya bisa pakai design editor, yang bisa di drag dan drop, lalu juga kompatibel dengan komponen2 lama. Sedangkan compose, pembuatannya lebih cepat. Lalu kodenya lebih konsisten dan mudah dibaca. Terakhir, tidak butuh ID karena langsung deklaratif. Sedangkan kekurangannya, Belum sepenuhnya stabil di semua skenario. lalu belum semua library support Compose.. Dan untuk project lama diperlukan migrasi bertahap